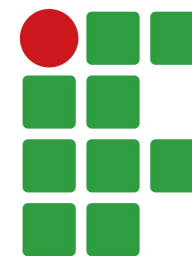


# PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software  
Lucas Sampaio Leite



**INSTITUTO  
FEDERAL**  
Pernambuco

# Sintaxe básica do Kotlin

- Estruturas de repetição: for

```
val numeros = intArrayOf(10, 20, 30, 40)

for (numero in numeros)
    println(numero)

for (indice in numeros.indices)
    println("numeros[$indice] = ${numeros[indice]}")
```



```
10
20
30
40
numeros[0] = 10
numeros[1] = 20
numeros[2] = 30
numeros[3] = 40
```

# Sintaxe básica do Kotlin

- Estruturas de repetição: for

```
val numeros = intArrayOf(10, 20, 30, 40)  
  
for ((indice, elemento) in numeros.withIndex())  
    println("numeros[$indice] = $elemento")
```



```
numeros[0] = 10  
numeros[1] = 20  
numeros[2] = 30  
numeros[3] = 40
```

# Sintaxe básica do Kotlin

- Estruturas de repetição: for

```
for (i in 1..10)  
    println(i)
```



```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

```
for (i in 10 downTo 1)  
    println(i)
```



```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1
```

# Sintaxe básica do Kotlin

- Estruturas de repetição: for

```
for (i in 1..10 step 2)  
    println(i)
```



1  
3  
5  
7  
9

```
for (i in 10 downTo 1 step 2)  
    println(i)
```



10  
8  
6  
4  
2

# Sintaxe básica do Kotlin

- Estruturas de repetição: `forEach` e `forEachIndexed`:
  - `forEach` permite percorrer uma coleção executando uma expressão lambda para cada elemento.
  - `forEachIndexed` permite acessar o índice e o elemento durante a iteração.

```
val numeros = intArrayOf(10, 20, 30, 40)

numeros.forEach{ println(it) }
numeros.forEachIndexed{ indice, valor -> println("$indice: $valor") }
```



```
10
20
30
40
0: 10
1: 20
2: 30
3: 40
```



# Sintaxe básica do Kotlin

- Estruturas de repetição: while

```
var contador = 1  
  
while (contador <= 5) {  
    println(contador)  
    contador++  
}
```



```
1  
2  
3  
4  
5
```

# Sintaxe básica do Kotlin

- Estruturas de repetição: do-while

```
var numero = 1  
  
do {  
    println(numero)  
    numero++  
} while (numero <= 5)
```



1  
2  
3  
4  
5



# Sintaxe básica do Kotlin

- Funções:
  - Em Kotlin, funções são definidas utilizando a palavra-chave `fun`.
  - Uma função é composta por nome, parâmetros e tipo de retorno.
  - O corpo da função é delimitado por chaves `{ }`.
- Definindo uma função:

```
fun soma(a: Int, b: Int): Int {  
    return a + b  
}
```

- Chamando uma função:

```
val resultado = soma(4,5)
```

# Sintaxe básica do Kotlin

- Funções:
  - Unit indica que a função não retorna um valor.
  - Quando o retorno é Unit, sua declaração pode ser omitida.

```
fun saudar(): Unit {  
    println("Olá!")  
}
```



```
fun saudar() {  
    println("Olá!")  
}
```

# Sintaxe básica do Kotlin

- Funções:
  - Unit indica que a função não retorna um valor.
  - Quando o retorno é Unit, sua declaração pode ser omitida.

```
fun saudar(): Unit {  
    println("Olá!")  
}
```



```
fun saudar() {  
    println("Olá!")  
}
```

Boa prática: omitir Unit quando a função não retorna um valor, tornando o código mais limpo e idiomático.

# Sintaxe básica do Kotlin

- Funções de expressão única (Single-Expression Functions):
  - São utilizadas quando a função possui apenas uma expressão.
  - Não é necessário utilizar chaves ({ }) nem a palavra-chave return.
  - O compilador pode inferir automaticamente o tipo de retorno.

```
fun soma(a: Int, b: Int) = a + b
```



```
fun soma(a: Int, b: Int): Int {  
    return a + b  
}
```

# Sintaxe básica do Kotlin

- Retorno Any:
  - Any é utilizado quando uma função puder retornar objetos de tipos diferentes.
  - Any é a superclasse de todas as classes em Kotlin, semelhante a Object em Java.
  - O tipo real do retorno pode ser verificado com is.

```
fun obterValor(opcao: Int): Any {  
    return when (opcao) {  
        1 -> "Kotlin"  
        2 -> 25  
        3 -> true  
        else -> 0.0  
    }  
}
```

```
val valor = obterValor(2)  
  
if (valor is Int) {  
    println(valor + 10)  
}
```

# Sintaxe básica do Kotlin

- Exemplos:

```
fun printMessage(message: String): Unit {                               // 1
    println(message)
}

fun printMessageWithPrefix(message: String, prefix: String = "Info") { // 2
    println("[$prefix] $message")
}

fun sum(x: Int, y: Int): Int {                                           // 3
    return x + y
}

fun multiply(x: Int, y: Int) = x * y                                     // 4

fun main() {
    printMessage("Hello")                                                // 5
    printMessageWithPrefix("Hello", "Log")                               // 6
    printMessageWithPrefix("Hello")                                     // 7
    printMessageWithPrefix(prefix = "Log", message = "Hello")           // 8
    println(sum(1, 2))                                                    // 9
    println(multiply(2, 4))                                              // 10
}
```

# Sintaxe básica do Kotlin

- Funções de escopo
  - Permitem executar um bloco de código no contexto de um objeto.
  - Tornam o código mais conciso e legível.
  - Evitam repetir o nome do objeto ao realizar várias operações.
  - Algumas das principais funções de escopo: `let`, `run` e `apply`.

# Sintaxe básica do Kotlin

- let e run:
  - Executam um bloco de código no contexto de um objeto.
  - Retornam o resultado da última expressão do bloco.
  - let acessa o objeto por meio de it (ou um nome definido pelo programador).
  - run acessa o objeto por meio de this, permitindo chamar seus membros diretamente.

```
val tamanho = "Kotlin".let {  
    println(it)  
    it.length  
}
```

```
val tamanho = "Kotlin".run {  
    println(this)  
    length  
}
```



# Sintaxe básica do Kotlin

- `apply`:
  - Executa um bloco de código no contexto de um objeto.
  - O objeto é acessado por meio de `this` (implícito).
  - Retorna o próprio objeto, permitindo encadear chamadas.
  - É muito utilizado para configurar ou inicializar objetos (setup).

```
data class Usuario(  
    var nome: String = "",  
    var idade: Int = 0,  
    var email: String = ""  
)
```

```
val usuario = Usuario().apply {  
    nome = "Lucas"  
    idade = 30  
    email = "lucas@email.com"  
}
```

# Sintaxe básica do Kotlin

- Parâmetros variáveis (vararg):
  - A palavra-chave vararg permite que uma função receba um número variável de argumentos do mesmo tipo.
  - Os argumentos são passados separadamente, utilizando vírgulas.
  - Dentro da função, os argumentos podem ser manipulados como um array.

```
fun printAll(vararg messages: String) {  
    for (m in messages) println(m)  
}  
printAll("Hello", "Hallo", "Salut", "Hola", "你好")  
  
fun printAllWithPrefix(vararg messages: String, prefix: String) {  
    for (m in messages) println(prefix + m)  
}  
printAllWithPrefix(  
    "Hello", "Hallo", "Salut", "Hola", "你好",  
    prefix = "Greeting: "  
)  
  
fun log(vararg entries: String) {  
    printAll(*entries)  
}  
log("Hello", "Hallo", "Salut", "Hola", "你好")
```

# Sintaxe básica do Kotlin

- Funções de ordem superior (Higher-Order Functions):
  - São funções que recebem outras funções como parâmetros e/ou retornam funções.
  - Permitem criar código mais flexível, reutilizável e expressivo.

```
fun calcular(a: Int, b: Int, operacao: (Int, Int) -> Int): Int {  
    return operacao(a, b)  
}  
  
fun main() {  
    val soma = calcular(10, 20) { x, y -> x + y }  
    println(soma)  
}
```

# Sintaxe básica do Kotlin

- Funções de ordem superior (Higher-Order Functions):

```
fun operacao(x: Int, y: Int, funcao: (Int, Int) -> Int): Int {  
    return funcao(x, y)  
}  
  
fun soma(a: Int, b: Int): Int {  
    return a + b  
}  
  
fun multiplicacao(a: Int, b: Int): Int {  
    return a * b  
}  
  
fun main() {  
    val resultadoSoma = operacao(10, 5, ::soma)  
    println("Resultado da soma: $resultadoSoma") // Saída: Resultado da soma: 15  
  
    val resultadoMultiplicacao = operacao(10, 5, ::multiplicacao)  
    println("Resultado da multiplicação: $resultadoMultiplicacao") // Saída: Resultado da multiplicação: 50  
  
    val resultadoDivisao = operacao(10, 2) { a, b -> a / b }  
    println("Resultado da divisão: $resultadoDivisao") // Saída: Resultado da divisão: 5  
}
```

# Sintaxe básica do Kotlin

- Funções de ordem superior (Higher-Order Functions):

```
fun criarMultiplicador(fator: Int): (Int) -> Int {  
    return { numero -> numero * fator }  
}  
  
fun main() {  
    val dobrar = criarMultiplicador(2)  
    val triplicar = criarMultiplicador(3)  
  
    println(dobrar(10))           // 20  
    println(triplicar(10))        // 30  
}
```



# Sintaxe básica do Kotlin

- Tratamento de valores nulos (Null Safety):
  - Kotlin possui um sistema de tipos que ajuda a evitar NullPointerException (NPE).
  - Por padrão, as variáveis são non-nullable, ou seja, não podem armazenar null.
  - Para permitir valores nulos, o tipo deve ser declarado como nullable, utilizando o sufixo “?”.

```
var nome: String = "Kotlin"  
nome = null // Erro de compilação
```



```
Null can not be a value of a non-null type String
```

# Sintaxe básica do Kotlin

- Tratamento de valores nulos (Null Safety):
  - Kotlin possui um sistema de tipos que ajuda a evitar NullPointerException (NPE).
  - Por padrão, as variáveis são non-nullable, ou seja, não podem armazenar null.
  - Para permitir valores nulos, o tipo deve ser declarado como nullable, utilizando o sufixo “?”.

```
var nome: String = "Kotlin"  
nome = null // Erro de compilação
```

Null can not be a value of a non-null type String

Solução:

```
var nome: String? = "Kotlin"  
nome = null // Permitido!
```

# Sintaxe básica do Kotlin

- Operador de chamada segura (?.)
  - Permite acessar propriedades ou chamar métodos apenas se o objeto não for nulo.
  - Se o objeto for null, a expressão retorna null e a execução não gera exceção.
  - É uma forma segura de acessar objetos nullable.

```
val nome: String? = null
val tamanho: Int? = nome?.length
println(tamanho) // Saída: null
```

```
val nome: String? = "Kotlin"
val tamanho: Int? = nome?.length
println(tamanho) // Saída: 6
```



# Sintaxe básica do Kotlin

```
fun buscarUsuario(id: Int): Usuario? {  
    // Simula uma consulta ao banco  
    return if (id == 1) {  
        Usuario("Lucas", "lucas@email.com")  
    } else {  
        null  
    }  
}  
  
fun main() {  
    val usuario = buscarUsuario(2)  
  
    println(usuario?.nome)  
    println(usuario?.email)  
}
```

# Sintaxe básica do Kotlin

- Operador Elvis (?:)
  - Permite definir um valor padrão quando uma expressão resultar em null.
  - Sintaxe: `val resultado = valor ?: valorPadrao`
  - Avalia a expressão à esquerda:
  - Se não for null, retorna o valor dessa expressão.
  - Se for null, retorna o valor definido à direita.

# Sintaxe básica do Kotlin

- Operador Elvis (?:)
  - Permite definir um valor padrão quando uma expressão resultar em null.
  - Sintaxe: `val resultado = valor ?: valorPadrao`
  - Avalia a expressão à esquerda:
  - Se não for null, retorna o valor dessa expressão.
  - Se for null, retorna o valor definido à direita.

```
val nome: String? = null
val nomeUsuario: String = nome ?: "Usuário desconhecido"
println(nomeUsuario) // Saída: Usuário desconhecido
```

# Sintaxe básica do Kotlin

```
fun buscarUsuario(id: Int): Usuario? {  
    return if (id == 1) {  
        Usuario("Lucas")  
    } else {  
        null  
    }  
}  
  
fun main() {  
    val usuario = buscarUsuario(2)  
  
    val nome = usuario?.nome ?: "Usuário não encontrado"  
  
    println(nome)  
}
```

# Sintaxe básica do Kotlin

- Operador de força nula (!!):
  - Permite informar ao compilador que uma expressão nullable não possui valor nulo.
  - Ao utilizar !!, o Kotlin trata a expressão como non-nullable.
  - Caso a expressão seja realmente null, uma NullPointerException será lançada em tempo de execução.

```
fun obterNome(): String? {  
    return null  
}  
fun main() {  
    val nome: String? = obterNome()  
    val comprimento: Int = nome!!.length  
    println("Comprimento do nome: $comprimento")  
}
```

# Sintaxe básica do Kotlin

- Operador de força nula (!!):
  - Permite informar ao compilador que uma expressão nullable não possui valor nulo.
  - Ao utilizar !!, o Kotlin trata a expressão como non-nullable.
  - Caso a expressão seja realmente null, uma NullPointerException será lançada em tempo de execução.

```
fun obterNome(): String? {  
    return null  
}  
fun main() {  
    val nome: String? = obterNome()  
    val comprimento: Int = nome!!.length  
    println("Comprimento do nome: $comprimento")  
}
```

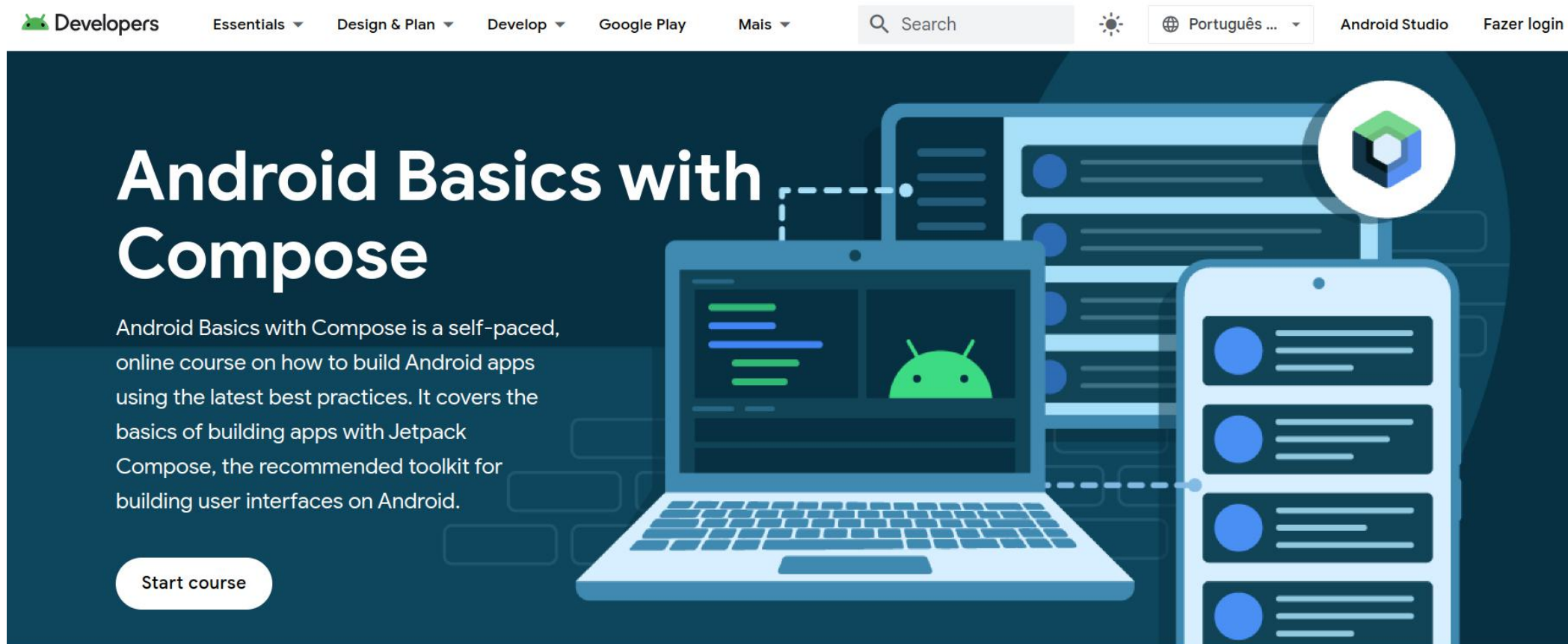
```
Exception in thread "main" java.lang.NullPointerException  
    at FileKt.main (File.kt:8)  
    at FileKt.main (File.kt:-1)  
    at jdk.internal.reflect.NativeMethodAccessorImpl.invoke0 (:-2)
```

# Sintaxe básica do Kotlin

- Boas práticas de Null Safety:
  - Prefira tipos não nulos (Non-Nullable) sempre que possível.
  - Evite o operador de força nula (!!).
  - Utilize o operador de chamada segura (?.).
  - Use o operador Elvis (?:) para definir valores padrão.



# Prática de laboratório



The screenshot shows the top navigation bar of the Android Developers website. It includes links for 'Developers', 'Essentials', 'Design & Plan', 'Develop', 'Google Play', and 'Mais'. There is a search bar, a language selector set to 'Português ...', and links for 'Android Studio' and 'Fazer login'. The main content area features a large illustration of a laptop and a smartphone, both displaying the Android logo. The text 'Android Basics with Compose' is prominently displayed, followed by a description of the course as a self-paced, online course on building Android apps using Jetpack Compose. A 'Start course' button is visible at the bottom left of the main content area.

Android Basics with Compose

Android Basics with Compose is a self-paced, online course on how to build Android apps using the latest best practices. It covers the basics of building apps with Jetpack Compose, the recommended toolkit for building user interfaces on Android.

Start course

Link: <https://developer.android.com/courses/android-basics-compose/course>

# Prática de laboratório

**PROGRAMA DE TREINAMENTOS 1**



**6 ATIVIDADES**

## Introdução ao Kotlin

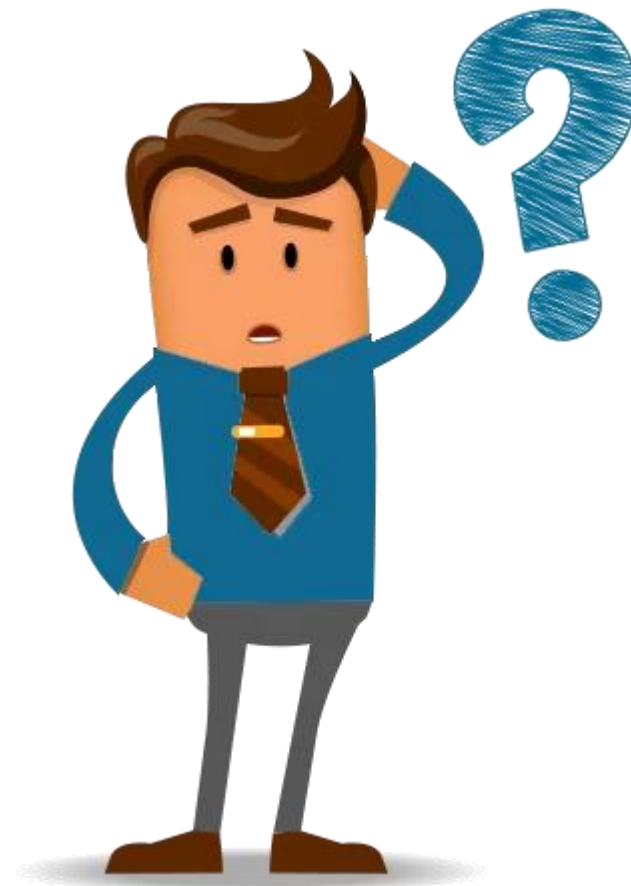
Aprenda conceitos introdutórios de programação em Kotlin para começar a criar apps Android nessa linguagem.

Abril de 2022

Confira

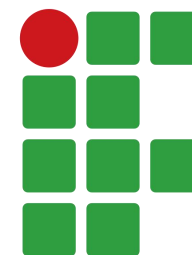
Link: <https://developer.android.com/courses/android-basics-compose/course>

# Dúvidas



# PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software  
Lucas Sampaio Leite



**INSTITUTO  
FEDERAL**  
Pernambuco